

Punto 3.2 - Consultas básicas: Where, OrderBy y ThenBy

Práctica

Ejercicio: Añadir métodos específicos al repositorio de órdenes para encapsular consultas con filtros y ordenaciones. Crear un caso de uso que ejecute estas consultas, analice el SQL generado y compare los resultados con diferentes combinaciones de filtros y ordenaciones.

Contexto del proyecto: En el punto 3.1 se exploraron los fundamentos de LINQ to Entities. En este punto se añaden consultas básicas con Where, OrderBy, ThenBy, OrderByDescending y ThenByDescending. Estas consultas se encapsulan en métodos del repositorio para evitar exponer IQueryable fuera de la infraestructura. Esta encapsulación se usará en el punto 3.3 para las proyecciones con Select.

Paso 1: Abrir el proyecto

```
cd AceriaData
cd src/AceriaData.Console
```

cd AceriaData → entra en la carpeta raíz del proyecto. cd src/AceriaData.Console → entra en la carpeta del proyecto de consola.

Paso 2: Añadir los métodos de consulta a la interfaz del repositorio

Modificar src/AceriaData.Application/Interfaces/IOrdenRepositorio.cs:

```
using AceriaData.Domain.Entities;

namespace AceriaData.Application.Interfaces;

public interface IOrdenRepositorio
{
    OrdenFabricacion? ObtenerPorId(int id);
    List<OrdenFabricacion> ObtenerTodas();
    IQueryable<OrdenFabricacion> ObtenerQueryable();
    OrdenFabricacion? ObtenerPorNumeroOrden(string numeroOrden);
    List<OrdenFabricacion> ObtenerPendientesPorCliente(string cliente);
    List<OrdenFabricacion> ObtenerPorEstadoOrdenadasPorFecha(string estado);
    List<OrdenFabricacion> ObtenerPorRangoDeFechas(DateTime desde, DateTime hasta);
    List<OrdenFabricacion> ObtenerPorClienteOrdenadas(string cliente);
    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
}
```

Línea 1: using AceriaData.Domain.Entities; → importa las entidades del dominio.

Línea 3: namespace AceriaData.Application.Interfaces; → declara el espacio de nombres.

Línea 5: public interface IOrdenRepositorio → declara la interfaz.

Línea 7: OrdenFabricacion? ObtenerPorId(int id); → método que obtiene por Id.

Línea 8: List ObtenerTodas(); → método que obtiene todas.

Línea 9: IQueryable ObtenerQueryable(); → método que devuelve IQueryable para el punto 3.1.

Línea 10: OrdenFabricacion? ObtenerPorNumeroOrden(string numeroOrden); → método que obtiene por número.

Línea 11: List ObtenerPendientesPorCliente(string cliente); → método que obtiene las órdenes pendientes de un cliente.

Línea 12: `List ObtenerPorEstadoOrdenadasPorFecha(string estado);` → método que obtiene las órdenes por estado ordenadas por fecha.

Línea 13: `List ObtenerPorRangoDeFechas(DateTime desde, DateTime hasta);` → método que obtiene las órdenes por rango de fechas.

Línea 14: `List ObtenerPorClienteOrdenadas(string cliente);` → método que obtiene las órdenes de un cliente ordenadas.

Línea 15: `void Agregar(OrdenFabricacion orden);` → método que agrega una orden.

Línea 16: `void Eliminar(OrdenFabricacion orden);` → método que elimina una orden.

Error común: si los métodos devuelven `IQueryable`, la capa de aplicación puede componer consultas que el repositorio no controla. Se deben devolver listas o entidades concretas.

| Paso 3: Implementar los métodos en el repositorio

Modificar `src/AceriaData.Infrastructure.Repositories/OrdenRepositorio.cs`:

```

using AceriaData.Application.Interfaces;
using AceriaData.Domain.Entities;
using AceriaData.Infrastructure.Persistence;

namespace AceriaData.Infrastructure.Repositories;

public class OrdenRepository : IOrdenRepository
{
    private readonly AceriaDbContext _context;

    public OrdenRepository(AceriaDbContext context)
    {
        _context = context;
    }

    public OrdenFabricacion? ObtenerPorId(int id)
    {
        return _context.OrdenesFabricacion.Find(id);
    }

    public List<OrdenFabricacion> ObtenerTodas()
    {
        return _context.OrdenesFabricacion.ToList();
    }

    public IQueryable<OrdenFabricacion> ObtenerQueryable()
    {
        return _context.OrdenesFabricacion.AsQueryable();
    }

    public OrdenFabricacion? ObtenerPorNumeroOrden(string numeroOrden)
    {
        return _context.OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == numeroOrden);
    }

    public List<OrdenFabricacion> ObtenerPendientesPorCliente(string cliente)
    {
        return _context.OrdenesFabricacion
            .Where(o => o.Cliente == cliente && o.Estado == "Pendiente")
            .OrderBy(o => o.FechaCreacion)
            .ToList();
    }

    public List<OrdenFabricacion> ObtenerPorEstadoOrdenadasPorFecha(string estado)
    {
        return _context.OrdenesFabricacion
            .Where(o => o.Estado == estado)
            .OrderByDescending(o => o.FechaCreacion)
            .ToList();
    }

    public List<OrdenFabricacion> ObtenerPorRangoDeFechas(DateTime desde, DateTime hasta)
    {
        return _context.OrdenesFabricacion
            .Where(o => o.FechaCreacion >= desde && o.FechaCreacion <= hasta)
            .OrderBy(o => o.FechaCreacion)
            .ToList();
    }

    public List<OrdenFabricacion> ObtenerPorClienteOrdenadas(string cliente)
    {
        return _context.OrdenesFabricacion
            .Where(o => o.Cliente == cliente)
            .OrderBy(o => o.Cliente)
            .ThenByDescending(o => o.FechaCreacion)
            .ToList();
    }

    public void Agregar(OrdenFabricacion orden)
    {
        _context.OrdenesFabricacion.Add(orden);
    }
}

```

```

public void Eliminar(OrdenFabricacion orden)
{
    _context.OrdenesFabricacion.Remove(orden);
}
}

```

Línea 1: using AceriaData.Application.Interfaces; → importa la interfaz del repositorio.

Línea 2: using AceriaData.Domain.Entities; → importa las entidades del dominio.

Línea 3: using AceriaData.Infrastructure.Persistence; → importa el DbContext.

Línea 5: namespace AceriaData.Infrastructure.Repositories; → declara el espacio de nombres.

Línea 7: public class OrdenRepositorio : IOrdenRepositorio → declara la implementación.

Línea 9: private readonly AceriaDbContext _context; → campo del DbContext.

Línea 11: public OrdenRepositorio(AceriaDbContext context) → constructor.

Línea 13: _context = context; → asigna el parámetro al campo.

Línea 16: public OrdenFabricacion? ObtenerPorId(int id) → método que obtiene por Id.

Línea 18: return _context.OrdenesFabricacion.Find(id); → busca por clave primaria.

Línea 21: public List ObtenerTodas() → método que obtiene todas.

Línea 23: return _context.OrdenesFabricacion.ToList(); → materializa la consulta.

Línea 26: public IQueryable ObtenerQueryable() → método que devuelve IQueryable.

Línea 28: return _context.OrdenesFabricacion.AsQueryable(); → devuelve la consulta sin materializar.

Línea 31: public OrdenFabricacion? ObtenerPorNumeroOrden(string numeroOrden) → método que obtiene por número.

Línea 33: return _context.OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == numeroOrden); → filtra por número y devuelve la primera coincidencia.

Línea 36: public List ObtenerPendientesPorCliente(string cliente) → método que obtiene las órdenes pendientes de un cliente.

Línea 38: return _context.OrdenesFabricacion → inicia la consulta.

Línea 39: .Where(o => o.Cliente == cliente && o.Estado == "Pendiente") → filtra por cliente y estado.

Línea 40: .OrderBy(o => o.FechaCreacion) → ordena por fecha ascendente.

Línea 41: .ToList(); → materializa la consulta.

Línea 44: public List ObtenerPorEstadoOrdenadasPorFecha(string estado) → método que obtiene las órdenes por estado ordenadas por fecha.

Línea 46: return _context.OrdenesFabricacion → inicia la consulta.

Línea 47: .Where(o => o.Estado == estado) → filtra por estado.

Línea 48: .OrderByDescending(o => o.FechaCreacion) → ordena por fecha descendente.

Línea 49: .ToList(); → materializa la consulta.

Línea 52: public List ObtenerPorRangoDeFechas(DateTime desde, DateTime hasta) → método que obtiene las órdenes por rango de fechas.

Línea 54: return _context.OrdenesFabricacion → inicia la consulta.

Línea 55: .Where(o => o.FechaCreacion >= desde && o.FechaCreacion <= hasta) → filtra por rango de fechas.

Línea 56: `.OrderBy(o => o.FechaCreacion)` → ordena por fecha ascendente.

Línea 57: `.ToList();` → materializa la consulta.

Línea 60: `public List ObtenerPorClienteOrdenadas(string cliente)` → método que obtiene las órdenes de un cliente ordenadas.

Línea 62: `return _context.OrdenesFabricacion` → inicia la consulta.

Línea 63: `.Where(o => o.Cliente == cliente)` → filtra por cliente.

Línea 64: `.OrderBy(o => o.Cliente)` → ordena por cliente ascendente.

Línea 65: `.ThenByDescending(o => o.FechaCreacion)` → añade un criterio secundario por fecha descendente.

Línea 66: `.ToList();` → materializa la consulta.

Línea 69: `public void Agregar(OrdenFabricacion orden)` → método que agrega una orden.

Línea 71: `_context.OrdenesFabricacion.Add(orden);` → registra la entidad.

Línea 74: `public void Eliminar(OrdenFabricacion orden)` → método que elimina una orden.

Línea 76: `_context.OrdenesFabricacion.Remove(orden);` → marca la entidad para eliminar.

Error común: si se llama a `ToList` antes de aplicar los filtros y las ordenaciones, se carga toda la tabla en memoria y se pierde la ventaja de la traducción a SQL.

| Paso 4: Crear el caso de uso de consultas básicas

Crear el archivo `src/AceriaData.Application/UseCases/ConsultasBasicasUseCase.cs`:

```
using AceriaData.Application.Interfaces;

namespace AceriaData.Application.UseCases;

public class ConsultasBasicasUseCase
{
    private readonly IUnidadDeTrabajo _unidad;

    public ConsultasBasicasUseCase(IUnidadDeTrabajo unidad)
    {
        _unidad = unidad;
    }

    public void Ejecutar()
    {
        Console.WriteLine("=== CONSULTAS BÁSICAS CON LINQ ===");

        DemostrarWhereSimple();
        DemostrarWhereCompuesto();
        DemostrarOrdenacionSimple();
        DemostrarOrdenacionMultiple();
        DemostrarCombinacion();
    }

    private void DemostrarWhereSimple()
    {
        Console.WriteLine("\n-- Where simple: órdenes pendientes del cliente 'Constructora del Norte'
---");

        var ordenes = _unidad.Ordenes.ObtenerPendientesPorCliente("Constructora del Norte");
        foreach (var orden in ordenes)
        {
            Console.WriteLine($" {orden.NumeroOrden} | {orden.Cliente} | {orden.Estado} |
{orden.FechaCreacion:dd/MM/yyyy}");
        }
    }

    private void DemostrarWhereCompuesto()
```

```

    {
        Console.WriteLine("\n--- Where compuesto: órdenes en estado 'Pendiente' ordenadas por fecha
descendente ---");

        var ordenes = _unidad.Ordenes.ObtenerPorEstadoOrdenadasPorFecha("Pendiente");
        foreach (var orden in ordenes)
        {
            Console.WriteLine($" {orden.NumeroOrden} | {orden.Cliente} | {orden.Estado} |
{orden.FechaCreacion:dd/MM/yyyy}");
        }
    }

    private void DemostrarOrdenacionSimple()
    {
        Console.WriteLine("\n--- Ordenación simple: órdenes por rango de fechas ---");

        var desde = new DateTime(2024, 1, 1);
        var hasta = new DateTime(2025, 12, 31);

        var ordenes = _unidad.Ordenes.ObtenerPorRangoDeFechas(desde, hasta);
        foreach (var orden in ordenes)
        {
            Console.WriteLine($" {orden.NumeroOrden} | {orden.Cliente} | {orden.FechaCreacion:dd/MM/
yyyy}");
        }
    }

    private void DemostrarOrdenacionMultiple()
    {
        Console.WriteLine("\n--- Ordenación múltiple: órdenes del cliente 'Constructora del Norte'
---");

        var ordenes = _unidad.Ordenes.ObtenerPorClienteOrdenadas("Constructora del Norte");
        foreach (var orden in ordenes)
        {
            Console.WriteLine($" {orden.NumeroOrden} | {orden.Cliente} | {orden.FechaCreacion:dd/MM/
yyyy}");
        }
    }

    private void DemostrarCombinacion()
    {
        Console.WriteLine("\n--- Combinación: SQL generado por una consulta con Where, OrderBy y ThenBy
---");

        var consulta = _unidad.Ordenes.ObtenerQueryable()
            .Where(o => o.Estado == "Pendiente")
            .OrderBy(o => o.Cliente)
            .ThenByDescending(o => o.FechaCreacion);

        var sql = consulta.ToQueryString();
        Console.WriteLine(sql);
    }
}

```

Línea 1: using AceriaData.Application.Interfaces; → importa las interfaces de la aplicación.

Línea 3: namespace AceriaData.Application.UseCases; → declara el espacio de nombres.

Línea 5: public class ConsultasBasicasUseCase → declara el caso de uso.

Línea 7: private readonly IUnidadDeTrabajo _unidad; → campo de la unidad de trabajo.

Línea 9: public ConsultasBasicasUseCase(IUnidadDeTrabajo unidad) → constructor.

Línea 11: _unidad = unidad; → asigna el parámetro al campo.

Línea 14: public void Ejecutar() → declara el método principal.

Línea 16: Console.WriteLine("=== CONSULTAS BÁSICAS CON LINQ ==="); → muestra la cabecera.

Línea 18: DemostrarWhereSimple(); → llama al método de Where simple.

Línea 19: `DemostrarWhereCompuesto();` → llama al método de Where compuesto.

Línea 20: `DemostrarOrdenacionSimple();` → llama al método de ordenación simple.

Línea 21: `DemostrarOrdenacionMultiple();` → llama al método de ordenación múltiple.

Línea 22: `DemostrarCombinacion();` → llama al método de combinación.

Línea 25: `private void DemostrarWhereSimple()` → declara el método.

Línea 27: `Console.WriteLine("\n--- Where simple: órdenes pendientes del cliente 'Constructora del Norte' ---");` → muestra la cabecera.

Línea 29: `var ordenes = _unidad Ordenes.ObtenerPendientesPorCliente("Constructora del Norte");` → llama al método del repositorio.

Línea 30: `foreach (var orden in ordenes)` → itera sobre las órdenes.

Línea 32: `Console.WriteLine($" {orden.NumeroOrden} | {orden.Cliente} | {orden.Estado} | {orden.FechaCreacion:dd/MM/yyyy}");` → muestra los datos.

Línea 35: `private void DemostrarWhereCompuesto()` → declara el método.

Línea 37: `Console.WriteLine("\n--- Where compuesto: órdenes en estado 'Pendiente' ordenadas por fecha descendente ---");` → muestra la cabecera.

Línea 39: `var ordenes = _unidad Ordenes.ObtenerPorEstadoOrdenadasPorFecha("Pendiente");` → llama al método del repositorio.

Línea 40: `foreach (var orden in ordenes)` → itera sobre las órdenes.

Línea 42: `Console.WriteLine($" {orden.NumeroOrden} | {orden.Cliente} | {orden.Estado} | {orden.FechaCreacion:dd/MM/yyyy}");` → muestra los datos.

Línea 45: `private void DemostrarOrdenacionSimple()` → declara el método.

Línea 47: `Console.WriteLine("\n--- Ordenación simple: órdenes por rango de fechas ---");` → muestra la cabecera.

Línea 49: `var desde = new DateTime(2024, 1, 1);` → declara la fecha desde.

Línea 50: `var hasta = new DateTime(2025, 12, 31);` → declara la fecha hasta.

Línea 52: `var ordenes = _unidad Ordenes.ObtenerPorRangoDeFechas(desde, hasta);` → llama al método del repositorio.

Línea 53: `foreach (var orden in ordenes)` → itera sobre las órdenes.

Línea 55: `Console.WriteLine($" {orden.NumeroOrden} | {orden.Cliente} | {orden.FechaCreacion:dd/MM/yyyy}");` → muestra los datos.

Línea 58: `private void DemostrarOrdenacionMultiple()` → declara el método.

Línea 60: `Console.WriteLine("\n--- Ordenación múltiple: órdenes del cliente 'Constructora del Norte' ---");` → muestra la cabecera.

Línea 62: `var ordenes = _unidad Ordenes.ObtenerPorClienteOrdenadas("Constructora del Norte");` → llama al método del repositorio.

Línea 63: `foreach (var orden in ordenes)` → itera sobre las órdenes.

Línea 65: `Console.WriteLine($" {orden.NumeroOrden} | {orden.Cliente} | {orden.FechaCreacion:dd/MM/yyyy}");` → muestra los datos.

Línea 68: `private void DemostrarCombinacion()` → declara el método.

Línea 70: `Console.WriteLine("\n--- Combinación: SQL generado por una consulta con Where, OrderBy y ThenBy ---");` → muestra la cabecera.

Línea 72: `var consulta = _unidad Ordenes.ObtenerQueryable()` → inicia la consulta.

Línea 73: `.Where(o => o.Estado == "Pendiente")` → filtra por estado.

Línea 74: `.OrderBy(o => o.Cliente)` → ordena por cliente ascendente.

Línea 75: `.ThenByDescending(o => o.FechaCreacion)`; → añade un criterio secundario por fecha descendente.

Línea 77: `var sql = consulta.ToQueryString()`; → obtiene el SQL.

Línea 78: `Console.WriteLine(sql)`; → imprime el SQL.

Error común: si el método `ObtenerQueryable` no está implementado en el repositorio, el código no compila. Se debe asegurar que la interfaz y la implementación lo incluyen.

| Paso 5: Registrar el caso de uso en el contenedor

Modificar `src/AceriaData.Console/Program.cs`:

```
services.AddScoped<ConsultasBasicasUseCase>();
```

Línea 1: `services.AddScoped()`; → registra el caso de uso con ciclo de vida `Scoped`.

Error común: si se registra con `Singleton`, se comparte la misma instancia del `DbContext` entre todos los ámbitos y se producen problemas de concurrencia.

| Paso 6: Llamar al caso de uso desde la consola

Modificar el método `Main`:

```
using (var scope = _provider.CreateScope())
{
    var useCase = scope.ServiceProvider.GetRequiredService<ConsultasBasicasUseCase>();
    useCase.Ejecutar();
}
```

Línea 1: `using (var scope = _provider.CreateScope())` → crea un ámbito.

Línea 3: `var useCase = scope.ServiceProvider.GetRequiredService()`; → resuelve el caso de uso.

Línea 4: `useCase.Ejecutar()`; → ejecuta el caso de uso.

Error común: si se resuelve el caso de uso desde el proveedor raíz en lugar de desde un ámbito, se lanza una excepción `InvalidOperationException`.

| Paso 7: Insertar datos de prueba

Asegurarse de que hay datos en la base de datos. Modificar el método `Main` para insertar órdenes de prueba:

```

using (var scope = _provider.CreateScope())
{
    var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
    context.Database.EnsureDeleted();
    context.Database.EnsureCreated();

    var ordenes = new List<OrdenFabricacion>
    {
        new OrdenFabricacion { NumeroOrden = "OF-2024-0001", Cliente = "Constructora del Norte", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 1, 15) },
        new OrdenFabricacion { NumeroOrden = "OF-2024-0002", Cliente = "Constructora del Sur", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 2, 20) },
        new OrdenFabricacion { NumeroOrden = "OF-2024-0003", Cliente = "Constructora del Norte", Estado = "EnProceso", FechaCreacion = new DateTime(2024, 3, 10) },
        new OrdenFabricacion { NumeroOrden = "OF-2024-0004", Cliente = "Constructora del Este", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 4, 5) },
        new OrdenFabricacion { NumeroOrden = "OF-2024-0005", Cliente = "Constructora del Norte", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 5, 12) }
    };

    context.OrdenesFabricacion.AddRange(ordenes);
    context.SaveChanges();
}

```

Línea 1: using (var scope = _provider.CreateScope()) → crea un ámbito.

Línea 3: var context = scope.ServiceProvider.GetRequiredService(); → resuelve el DbContext.

Línea 4: context.Database.EnsureDeleted(); → elimina la base.

Línea 5: context.Database.EnsureCreated(); → crea la base.

Línea 7: var ordenes = new List → crea la lista de órdenes.

Línea 9: new OrdenFabricacion { ... }, → primera orden.

Línea 10: new OrdenFabricacion { ... }, → segunda orden.

Línea 11: new OrdenFabricacion { ... }, → tercera orden.

Línea 12: new OrdenFabricacion { ... }, → cuarta orden.

Línea 13: new OrdenFabricacion { ... } → quinta orden.

Línea 16: context.OrdenesFabricacion.AddRange(ordenes); → registra las órdenes.

Línea 17: context.SaveChanges(); → inserta las órdenes.

Error común: si se insertan las órdenes con el mismo número, la clave alternativa AK_OrdenesFabricacion_NumeroOrden rechaza la inserción.

| Paso 8: Ejecutar el proyecto

```
dotnet run
```

dotnet run → compila y ejecuta el proyecto.

Resultado esperado: aparecen las secciones con los resultados de cada consulta. Se observan los filtros por cliente y estado, las ordenaciones ascendentes y descendentes, y el SQL generado por la combinación de Where, OrderBy y ThenBy.

| Paso 9: Analizar la salida

La salida del programa muestra información como la siguiente:

=== CONSULTAS BÁSICAS CON LINQ ===

--- Where simple: órdenes pendientes del cliente 'Constructora del Norte' ---

```
OF-2024-0001 | Constructora del Norte | Pendiente | 15/01/2024
OF-2024-0005 | Constructora del Norte | Pendiente | 12/05/2024
```

--- Where compuesto: órdenes en estado 'Pendiente' ordenadas por fecha descendente ---

```
OF-2024-0005 | Constructora del Norte | Pendiente | 12/05/2024
OF-2024-0004 | Constructora del Este | Pendiente | 05/04/2024
OF-2024-0002 | Constructora del Sur | Pendiente | 20/02/2024
OF-2024-0001 | Constructora del Norte | Pendiente | 15/01/2024
```

--- Ordenación simple: órdenes por rango de fechas ---

```
OF-2024-0001 | Constructora del Norte | 15/01/2024
OF-2024-0002 | Constructora del Sur | 20/02/2024
OF-2024-0003 | Constructora del Norte | 10/03/2024
OF-2024-0004 | Constructora del Este | 05/04/2024
OF-2024-0005 | Constructora del Norte | 12/05/2024
```

--- Ordenación múltiple: órdenes del cliente 'Constructora del Norte' ---

```
OF-2024-0005 | Constructora del Norte | 12/05/2024
OF-2024-0003 | Constructora del Norte | 10/03/2024
OF-2024-0001 | Constructora del Norte | 15/01/2024
```

--- Combinación: SQL generado por una consulta con Where, OrderBy y ThenBy ---

```
SELECT [o].[Id], [o].[NumeroOrden], [o].[Cliente], [o].[FechaCreacion], [o].[Estado]
FROM [OrdenesFabricacion] AS [o]
WHERE [o].[Estado] = N'Pendiente'
ORDER BY [o].[Cliente], [o].[FechaCreacion] DESC
```

La primera sección muestra las órdenes pendientes del cliente "Constructora del Norte". La segunda sección muestra las órdenes pendientes ordenadas por fecha descendente. La tercera sección muestra las órdenes ordenadas por fecha ascendente. La cuarta sección muestra las órdenes del cliente "Constructora del Norte" ordenadas por cliente y fecha descendente. La quinta sección muestra el SQL generado.

Observaciones: el SQL generado incluye la cláusula WHERE con el filtro por estado y la cláusula ORDER BY con dos columnas. El orden de las cláusulas es el correcto: SELECT, FROM, WHERE, ORDER BY.

Paso 10: Diagnosticar un error común

Modificar el método ObtenerPendientesPorCliente para usar ToList antes del filtro:

```
csharp
public List<OrdenFabricacion> ObtenerPendientesPorCliente(string cliente)
{
    return _context.OrdenesFabricacion
        .ToList()
        .Where(o => o.Cliente == cliente && o.Estado == "Pendiente")
        .OrderBy(o => o.FechaCreacion)
        .ToList();
}
```

Resultado esperado: el resultado es el mismo, pero la consulta SQL ejecutada carga todas las órdenes en memoria y después filtra. Si la tabla tuviera un millón de filas, la consulta sería muy lenta.

Solución: aplicar ToList solo al final, después de todos los filtros y ordenaciones.

Errores comunes del ejercicio Error Causa Solución Filtro se aplica en memoria Se llamó a ToList antes del filtro Aplicar ToList al final Ordenación no aplicada Se llamó a OrderBy después de ToList Aplicar OrderBy antes de ToList Múltiples consultas Se materializó varias veces Materializar una sola vez al final IQueryable expuesto en el repositorio Anti-patrón Encapsular en métodos específicos (Módulo 4) Filtro por fecha incorrecto Se usó == en lugar de >= y <= Usar operadores de comparación ThenBy sin OrderBy Se llamó a ThenBy sin OrderBy previo Llamar a OrderBy primero Falta el using de EF Core El método ToQueryString no está disponible Añadir using Microsoft.EntityFrameworkCore;

Reto resuelto: Consulta con filtro por rango de fechas y ordenación múltiple

Reto: Crear un método en el repositorio que obtenga las órdenes de un cliente en un rango de fechas, ordenadas por estado ascendente y fecha de creación descendente. Añadir el método a la interfaz, implementarlo y usarlo desde el caso de uso.

Solución paso a paso:

Paso 1: Añadir el método a la interfaz IOrdenRepositorio:

```
List<OrdenFabricacion> ObtenerPorClienteYRangoDeFechas(string cliente, DateTime desde, DateTime hasta);
```

Línea 1: List ObtenerPorClienteYRangoDeFechas(string cliente, DateTime desde, DateTime hasta); → declara el método.

Paso 2: Implementar el método en OrdenRepositorio:

```
public List<OrdenFabricacion> ObtenerPorClienteYRangoDeFechas(string cliente, DateTime desde, DateTime hasta)
{
    return _context.OrdenesFabricacion
        .Where(o => o.Cliente == cliente && o.FechaCreacion >= desde && o.FechaCreacion <= hasta)
        .OrderBy(o => o.Estado)
        .ThenByDescending(o => o.FechaCreacion)
        .ToList();
}
```

Línea 1: public List ObtenerPorClienteYRangoDeFechas(string cliente, DateTime desde, DateTime hasta) → declara el método.

Línea 3: return _context.OrdenesFabricacion → inicia la consulta.

Línea 4: .Where(o => o.Cliente == cliente && o.FechaCreacion >= desde && o.FechaCreacion <= hasta) → filtra por cliente y rango de fechas.

Línea 5: .OrderBy(o => o.Estado) → ordena por estado ascendente.

Línea 6: .ThenByDescending(o => o.FechaCreacion) → añade un criterio secundario por fecha descendente.

Línea 7: .ToList(); → materializa la consulta.

Paso 3: Usar el método desde el caso de uso:

```
private void DemostrarConsultaCombinada()
{
    Console.WriteLine("\n--- Consulta combinada: cliente y rango de fechas ---");

    var desde = new DateTime(2024, 1, 1);
    var hasta = new DateTime(2024, 12, 31);

    var ordenes = _unidad.Ordenes.ObtenerPorClienteYRangoDeFechas("Constructora del Norte", desde, hasta);
    foreach (var orden in ordenes)
    {
        Console.WriteLine($" {orden.NumeroOrden} | {orden.Cliente} | {orden.Estado} | {orden.FechaCreacion:dd/MM/yyyy}");
    }
}
```

Línea 1: private void DemostrarConsultaCombinada() → declara el método.

Línea 3: Console.WriteLine("\n--- Consulta combinada: cliente y rango de fechas ---"); → muestra la cabecera.

Línea 5: `var desde = new DateTime(2024, 1, 1);` → declara la fecha desde.

Línea 6: `var hasta = new DateTime(2024, 12, 31);` → declara la fecha hasta.

Línea 8: `var ordenes = _unidad.Ordenes.ObtenerPorClienteYRangoDeFechas("Constructora del Norte", desde, hasta);` → llama al método del repositorio.

Línea 9: `foreach (var orden in ordenes)` → itera sobre las órdenes.

Línea 11: `Console.WriteLine($" {orden.NumeroOrden} | {orden.Cliente} | {orden.Estado} | {orden.FechaCreacion:dd/MM/yyyy}");` → muestra los datos.

Paso 4: Llamar al método desde Ejecutar:

```
DemostrarConsultaCombinada();
Paso 5: Ejecutar dotnet run y verificar que las órdenes se muestran ordenadas por estado y fecha descendente.
```

Resultado esperado: las órdenes del cliente "Constructora del Norte" en el rango de fechas se muestran ordenadas por estado ascendente y, dentro de cada estado, por fecha descendente.

| Analogía final

Las consultas básicas con LINQ en una acería son como las órdenes de búsqueda que el jefe de planta envía al archivo central. Where es el filtro: "solo quiero las órdenes del cliente Constructora del Norte". OrderBy es el criterio de ordenación: "ordénalas por fecha de creación". ThenBy es el criterio secundario: "y dentro de cada fecha, ordénalas por estado". El archivo central recibe la orden, busca en sus índices y devuelve exactamente lo que se pidió. Si el jefe pidiera todos los documentos y los ordenara él mismo, perdería tiempo y esfuerzo. Eso es lo que pasa cuando se materializa antes de filtrar. Las consultas encapsuladas en el repositorio son como las ventanillas especializadas del archivo: cada una sabe qué buscar y cómo ordenarlo. El jefe no necesita conocer el archivo por dentro: solo pide lo que necesita a la ventanilla correspondiente. Así funcionan las consultas básicas con LINQ: se filtran y se ordenan en el servidor, y los resultados se devuelven ya preparados.

| Resultado esperado

Al final del ejercicio, deberías haber:

Añadido los métodos de consulta a la interfaz `IOrdenRepositorio`.

Implementado los métodos en `OrdenRepositorio`.

Creado el caso de uso `ConsultasBasicasUseCase`.

Registrado el caso de uso en el contenedor.

Insertado datos de prueba.

Ejecutado las demostraciones de `Where`, `OrderBy`, `ThenBy`, `OrderByDescending` y `ThenByDescending`.

Analizado el SQL generado por una consulta combinada.

Diagnosticado el error de aplicar `ToList` antes del filtro.

Creado el método `ObtenerPorClienteYRangoDeFechas` y su demostración.

| Conexión con el siguiente punto

En este punto se han añadido consultas básicas con `Where`, `OrderBy`, `ThenBy`, `OrderByDescending` y `ThenByDescending` al proyecto `AceriaData`. Las consultas se han encapsulado en métodos del repositorio para evitar exponer `IQueryable` fuera de la infraestructura. En el siguiente punto se estudiarán las proyecciones con `Select`, incluyendo tipos anónimos, DTOs y proyecciones parciales.